TESA Top Gun Rally 2026 · Track 3 · อบรมวันที่ 18 พ.ย.

ให้ agent ขายแทน

จากกฎ Python สู่ LLM แล้วต่อยอดด้วย starter agent

09:00–12:00 · ช่วงบ่ายแข่งรอบเก็บคะแนนที่ 2 ห้ามส่งด้วยมือ

เช้านี้

สร้าง agent ทีละชั้น

09:00	ทำไมต้องมี agent · กติกาใหม่ · แนวคิด AI agent
09:15	**ลงมือ** agent ด้วยกฎ Python (notebook ข้อ 1)
09:45	LLM ทำงานอย่างไร · prompt · context · feedback loop
10:05	**ลงมือ** ใส่ LLM ในลูป (notebook ข้อ 2–3)
10:35	พัก
10:45	**ลงมือ** รัน starter agent และอ่าน log
11:15	**ลงมือ** แก้ strategy.py 1 จุด แล้วเทียบผล
11:45	กติกาช่วงบ่าย · เช็กลิสต์ก่อนแข่ง

ทำไมต้องมี agent

ตั้งแต่บ่ายนี้ คนห้ามส่งคำสั่งเอง

เมื่อวาน

คนอ่าน RFQ คิดราคา กดส่งเอง

ตลาดส่วนตัว 6 รอบ ผู้ซื้อ 2 แบบ

วันนี้และวันแข่ง

โปรแกรมตัดสินใจและส่งเองทุกรอบ

ตลาดรวม ทุกทีมแข่งกันสด ผู้ซื้อ 3 แบบ

คนยังแก้โค้ด แก้ prompt และรีสตาร์ท agent ได้ตลอด · แต่ทุกข้อเสนอต้องมาจาก agent

ทำไมต้องมี agent

ตลาดวันนี้ยากขึ้น 4 เรื่อง

ผู้ซื้อ Specialty

จ่ายสูงสุด แต่สเปกเข้มที่สุด รวมขนาดตะแกรงขั้นต่ำ

ตัวอย่างคลาดเคลื่อน

TV ที่คำนวณเป็นค่าประมาณ · ล็อตใกล้ขอบสเปกอาจโดน SPEC_MISMATCH

จำกัดกระสอบต่อรอบ

เสนอรวมได้ราว 1.5 เท่าของส่วนเฉลี่ยต่อรอบ · ดู **capacity_left_kg**

แข่งกับ 56 ทีมจริง

ทุกทีมได้โจทย์ล็อตเดียวกัน ความคลาดของตัวอย่างเท่ากัน · แย่ง RFQ ชุดเดียวกัน · มีรอบซ้อมก่อน

แนวคิด AI Agent

agent = โปรแกรมที่ดู คิด แล้วลงมือเอง

เป้าหมาย
ขายได้ราคาดี (PR สูง) และประเมินแม่น

รับรู้
อ่านสถานะตลาด ล็อต RFQ ผลรอบ
GET /v1/state

สมอง
ตัดสินใจว่าจะขายอะไร ให้ใคร ราคาเท่าไร · กฎ หรือ LLM

ลงมือ
เรียกเครื่องมือ (tool) ที่มี
POST /v1/offers:batch

ความจำ
เก็บผลแต่ละรอบไว้ใช้ตัดสินใจรอบหน้า

chatbot
คนถาม → ตอบครั้งเดียว → คนเอาไปทำต่อเอง

agent
วนดูผล → ตัดสินใจ → ลงมือเอง ทุกรอบ ไม่ต้องรอคน

แนวคิด AI Agent

สมองของ agent มีได้ 3 แบบ

กฎล้วน

if/else ที่คนเขียน

+ เร็ว แน่นอน ตรวจได้

– ไม่ปรับตัวเองตามสถานการณ์ใหม่

กฎ + LLM (ใช้วันนี้)

LLM เสนอ · Python ตรวจและลงมือ

+ ยืดหยุ่น แต่ตัวเลขยังปลอดภัย

– ต้องเขียนทั้งสองส่วน

LLM ล้วน

LLM เลือกเองทุกอย่าง แล้วส่งเลย

+ เขียนโค้ดน้อย

– คิดเลขผิด ช้า ตอบแปลก แล้วไม่มีใครกัน

หลักของงานนี้: **LLM ช่วยคิด · Python คุมตัวเลขและการส่ง** · LLM ล่มเมื่อไร กฎทำงานต่อได้ทันที

agent คือลูปที่ไม่มีวันหยุด



ทำตาม phase: APPRAISAL → ส่งผลประเมินครั้งเดียว · ROUND_OPEN รอบใหม่ → ตัดสินใจครั้งเดียวต่อรอบ · CLOSED → หยุด

agent ด้วยกฎ

สมองแบบกฎ: สิ่งที่เมื่อวานทำด้วยมือ

```
MARKUP = {"SPECIALTY": 1.00,  # ตัวคูณราคา
  "COMMERCIAL": 1.00, "INSTANT": 1.00}

for q in RFQ เรียงตาม MARKUP ของผู้ซื้อ สูงก่อน:
  for lot in ล็อต เรียงตาม TV สูงก่อน:
    ถ้าผ่านสเปก:
      kg = min(เหลือ, RFQ ต้องการ, capacity)
      ราคา = MARKUP[ประเภท] × TV
```

ข้อดี

เร็ว ไม่พลาด ตรวจได้ทุกบรรทัด

ข้อจำกัด

ทำแต่ที่เขียนไว้ ไม่ปรับตามผลเอง

เรียงตาม MARKUP = ผู้ซื้อที่เราตั้งราคาสูงได้ล็อตดีก่อน · ตอนนี้ตั้ง 1.00 เท่ากันหมด ลำดับจึงยังไม่เปลี่ยน จนกว่าจะปรับตัวคูณ

วันที่ 18 · 8 · notebook ข้อ 1.2 ฟังก์ชัน decide_rules()

ลงมือ · 30 นาที

notebook ข้อ 0–1: agent ตัวแรกของทีม

1. เปิด **สร้าง-agent.ipynb** ใน VS Code ใส่ที่อยู่ตลาดฝึกและ key ในข้อ 0
2. ข้อ 1.2 เดินเกมทีละขั้น: เริ่มเกม → **ประเมินราคา** → รอรอบเปิด → ตัดสินใจ → ส่ง → อ่านผล
3. ข้อ 1.3 ให้ลูปทำขั้นเหล่านี้ซ้ำเอง 3 รอบ ดูผลแต่ละรอบ
4. แก้ **MARKUP** ให้แต่ละประเภทผู้ซื้อไม่เท่ากัน แล้วรันใหม่

> ผลที่บอกทาง: ABOVE_LIMIT = แพงไป · OUTBID = คนอื่นคุ้มกว่า · ACCEPTED ตลอด = อาจถูกไป

ใส่ LLM · แนวคิด

LLM ทำงานอย่างไร (ฉบับคนใช้งาน)

เดาคำถัดไปทีละ token

token คือชิ้นส่วนของคำ · ตอบยาว = ช้า · ข้อความไทยมักใช้ token มากกว่าอังกฤษ

ไม่มีความจำระหว่างการเรียก

เรียกแต่ละครั้งเริ่มจากศูนย์ · อยากให้รู้อะไร ต้องส่งไปในข้อความทุกครั้ง

ไม่รู้จักตลาดของเรา

รู้แค่ความรู้ทั่วไปตอนฝึกโมเดล · กติกา ล็อต ผลรอบ ต้องบอกเองใน prompt

ผิดได้อย่างมั่นใจ

คิดเลขพลาด แต่งข้อมูลขึ้นเอง (hallucination) · ใช้ความเห็นได้ แต่ตัวเลขต้องให้ Python ตรวจ

ใส่ LLM

LLM ช่วยตัดสินใจ Python คุมตัวเลข

ให้ LLM ทำ

อ่านผลรอบที่แล้ว แล้วเสนอตัวคูณใหม่

เลือกว่ารอบนี้ควรขายเยอะหรือเก็บไว้

อธิบายเหตุผลสั้นๆ ลง log

ให้ Python ทำ

คำนวณ TV ตรวจสเปก นับกระสอบ

จำกัดตัวคูณให้อยู่ในช่วงที่ปลอดภัย

ส่งคำสั่ง จัดการ error ลองใหม่

ใส่ LLM · แนวคิด

prompt มี 2 ส่วน: system กับ user

system · คงที่ เขียนครั้งเดียว

บทบาท · กติกาตลาด · ความหมายของผล · รูปแบบคำตอบ

```
You set prices for a coffee farm in a
sealed-offer market. price = markup x
the lot's true value. ABOVE_LIMIT = too
expensive for that buyer ... Answer only JSON.
```

user · สร้างใหม่ทุกรอบ

ข้อมูลรอบนี้ · ตัวเลือกที่ Python กรองแล้ว · ผลรอบก่อน · คำสั่ง

```
Round R4 of 8.
Current markups: {"SPECIALTY": 1.00, ...}
My recent results:
R3: SPECIALTY x1.60 ABOVE_LIMIT ...
Choose the markups for this round.
```

แยกแบบนี้แก้ง่าย: ปรับนิสัยการคิดที่ system · ปรับข้อมูลที่ส่งที่ user · คำสั่งสั้น ชัด ภาษาอังกฤษ ตัวเลขคำนวณมาจาก Python

วันที่ 18 · 12 · notebook ข้อ 3 (SYSTEM, decide_llm) · starter: strategy.py (SYSTEM, llm_plan)

ใส่ LLM

บังคับให้ตอบเป็น JSON ที่โค้ดอ่านได้

```
r = llm.chat.completions.create(
  model="event-llm", max_tokens=400,
  messages=[{"role": "system", "content": SYSTEM},
            {"role": "user", "content": ผลรอบที่แล้ว}],
  response_format={"type": "json_schema", "json_schema": SCHEMA},
  extra_body={"reasoning_effort": "none"})
plan = json.loads(r.choices[0].message.content)
```

ให้ LLM เห็นผลจริงของรอบก่อน (สถานะ ราคา ตัวคูณ) · ไม่มีข้อมูล = ตัวเลขมั่ว

ใส่ LLM · แนวคิด

context: คำถามกับคำตอบต้องพอในแผ่นเดียว

gateway ของงาน: context รวม **8,192 token** ต่อการเรียกหนึ่งครั้ง



สีส้มซ้ายสุด = system ≈ 150 · ตัวเลขจากการทดสอบ starter บนเครื่องงานจริง: prompt ราว 2,200 token และยาวถึง 2,600 ตอนท้ายเกม

ล้นแล้วเป็นอย่างไร

gateway ตอบ error · agent ใช้กฎแทนทั้งรอบ

ส่งสรุป ไม่ส่งดิบ

ตัดช่องที่ไม่ใช้ · ผลแค่ 3 รอบล่าสุด · ตัวเลขปัดทศนิยม

ยาวขึ้น = ช้าขึ้น

ทุกทีมใช้ GPU ร่วมกัน · prompt สั้นได้คำตอบทันรอบกว่า

วันที่ 18 · 14 · ดู token ที่ใช้จริงได้จาก r.usage (notebook ข้อ 2)

ใส่ LLM · แนวคิด

feedback loop: เรียนจากผลรอบก่อน

ยื่นข้อเสนอ
ตัวคูณที่ LLM เลือก

→

ตลาดตอบผล
ACCEPTED · ABOVE_LIMIT · OUTBID

→

Python จำและสรุป
history / memory.py

→

ส่งให้ LLM
ใน user prompt รอบหน้า

```
# สิ่งที่ LLM เห็นในรอบ R4 (สรุปจาก memory.summary())
R2: my offers [COMMERCIAL x0.90 ACCEPTED; INSTANT x0.90 OUTBID]
R3: my offers [SPECIALTY x1.60 ABOVE_LIMIT]; market: SPECIALTY bought 80% of wanted
```

LLM ไม่ได้ "เรียนรู้" เอง · agent เก่งขึ้นได้เพราะ **Python เลือกว่าจะจำอะไร แล้วสรุปส่งให้** ทุกรอบ

วันที่ 18 · 15 · notebook ข้อ 1.3 history · starter: memory.py summary()

ใส่ LLM · แนวคิด

ปุ่มปรับ 4 ตัวที่ใช้บ่อย

ปุ่ม	ทำอะไร	ค่าใน starter
temperature	ความสุ่มของคำตอบ · 0 = ตอบเหมือนเดิม สูง = หลากหลาย	0.7
max_tokens	คำตอบยาวได้สูงสุดกี่ token · น้อยไป JSON ขาดกลางทาง	1,500
reasoning_effort	ให้คิดก่อนตอบ (thinking) · ฉลาดขึ้นบ้าง แต่ช้าและกิน token	ปิด (LLM_THINKING=0)
timeout	รอได้นานแค่ไหน ต้องน้อยกว่าเวลาที่เหลือในรอบ	เวลาที่เหลือ – 5 วิ (ไม่เกิน 60)

ปรับทีละปุ่ม แล้วดูใน log ว่าเร็วขึ้น ตอบดีขึ้น หรือพลาดบ่อยขึ้น · คำตอบที่ช้ากว่ารอบ = ไม่มีประโยชน์

วันที่ 18 · 16 · ตั้งได้ใน starter/.env (LLM_THINKING, LLM_TIMEOUT) และ llm.py

ใส่ LLM

LLM พังได้ agent ต้องไม่พังตาม

ถ้าเรียก LLM แล้วโปรแกรมหยุด agent จะไม่ส่งข้อเสนอเลยจนจบเกม · LLM ช่วยให้ขายดีขึ้น แต่ต้องขายได้แม้ไม่มี LLM

พลาดแบบ	เห็นอะไรใน Python	เกิดจาก	agent ทำอย่างไร
ช้าเกินรอบ	APITimeoutError	หลายทีมเรียกพร้อมกัน คำตอบยาว เปิด thinking	ตั้ง timeout ทุกครั้ง = เวลาที่เหลือในรอบ – เวลาส่งข้อเสนอ
เรียกถี่เกิน	RateLimitError (429)	เกินโควตาที่ gateway ให้แต่ละทีม	ถามรอบละครั้ง · ห้ามวนถามซ้ำทันที
key ผิด / ต่อไม่ได้	401 · APIConnectionError	ใส่ key หรือที่อยู่ gateway ผิด · เครือข่ายหลุด	ทดสอบกับตลาดฝึกก่อนแข่ง · ระหว่างแข่งใช้กฎไปก่อน
JSON ขาด/ผิด	JSONDecodeError	max_tokens น้อยไป คำตอบถูกตัด (finish = length)	ห่อด้วย try · คืน None แทนการ throw
ตัวเลขเพี้ยน	ไม่มี error เลย	LLM ไม่รู้กติกา ตอบตัวคูณ 5.0 หรือกระสอบเกินที่มี	guard ตรวจทุกค่า บีบให้อยู่ในช่วงที่ปลอดภัย



วันที่ 18 · 17 · notebook ข้อ 2.2 ask_json() ที่ไม่เคย throw และจำลอง timeout 0.5 วินาที

ลงมือ · 30 นาที

notebook ข้อ 2–3: ให้ LLM ปรับตัวคูณ

1. ข้อ 2: ใส่ที่อยู่ LLM gateway แล้วถามคำถามแรก
2. ข้อ 2.1–2.2: ขอคำตอบเป็น JSON และดูทางสำรองทำงาน
3. ข้อ 3: รัน **run(decide_llm)** ดูว่า LLM เลือกตัวคูณอะไร เพราะอะไร
4. เทียบกับ **decide_rules** ที่ทีมปรับไว้ในข้อ 1

> ตัวคูณจาก LLM ถูกบีบให้อยู่ใน 0.85–1.30 เสมอ · ปรับช่วงนี้ได้ในโค้ด



พัก 10 นาที

กลับมา 10:45 · ต่อด้วย starter ที่ใช้แข่งจริง

ระหว่างพัก: เปิดโฟลเดอร์ **starter** ใน VS Code ไว้ก่อน

starter สำหรับแข่ง

starter = notebook ที่ทนทานพร้อมแข่ง

main.py
ลูป · phase · ลองใหม่

→

candidates
คู่ล็อต-RFQ ที่ผ่านสเปก

→

llm_plan
LLM เลือกข้อเสนอ

→

กฎเติม
ขายส่วนที่เหลือ

→

guard
ตรวจก่อนส่ง batch

ทีมแก้แค่ **strategy.py** ก็พอ · ไฟล์อื่นดูแลการเชื่อมต่อให้แล้ว

เลือก RFQ ไหนก่อน · วิธีที่ 1 จาก 2

ผู้ซื้อที่จ่ายแพง ได้ของดีก่อน

```
# RFQ ของประเภทที่ตั้งตัวคูณสูงก่อน
for q in sorted(rfqs, key=lambda q: -markup[q["buyer_type"]]):
  # ล็อต TV สูงก่อน
  for lot in sorted(lots, key=lambda l: -tv[l["lot_id"]]):
    if passes(lot["sample"], q["spec"]):
      ยื่นซอง ราคา = markup × TV
      break  # RFQ ละ 1 ซอง
```

คิดแบบนี้

กาแฟมีจำกัด เอาล็อตดีไปเสนอผู้ซื้อที่ยอมจ่ายแพงก่อน ที่เหลือค่อยขายรายอื่น

ตัวอย่าง

ตั้งตัวคูณ Specialty > Commercial > Instant → วน RFQ ของ Specialty ก่อน แล้ว Commercial แล้ว Instant

จุดอ่อน: ถ้าทุกทีมคิดแบบนี้ ทุกทีมยื่น RFQ เดียวกันพร้อมกัน ส่วนใหญ่แพ้ (OUTBID) · ตั้งตัวคูณเท่ากันหมด (1.00) = ไม่ได้เรียงอะไรเลย

เลือก RFQ ไหนก่อน · วิธีที่ 2 จาก 2

สุ่มลำดับ แต่ RFQ ใหญ่มีโอกาสมาก่อน

```
# candidates(): สุ่ม u ระหว่าง 0–1 แล้ว
key = rng.random() ** (1 / rfq_bags)
เรียงตาม key มากไปน้อย

# u = 0.5 เท่ากัน:
#   ขอ 2 กระสอบ  → 0.5 ** (1/2)  = 0.71
#   ขอ 30 กระสอบ → 0.5 ** (1/30) = 0.98 ← มาก่อน

# rule_plan(): แล้วไล่ล็อต TV สูงก่อน
```

ทำไมสุ่ม

ทุกทีมเริ่มจาก starter เดียวกัน ถ้าเรียงตายตัว ทุกทีมจะแย่ง RFQ แรก สุ่มแล้วแต่ละทีมกระจายกันไปเอง

ทำไม RFQ ใหญ่

รับได้หลายกระสอบ หลายทีมขายได้พร้อมกัน · โอกาสมาก่อนเป็นสัดส่วนกับจำนวนกระสอบ

จุดอ่อน: ไม่สนว่าใครจ่ายแพง · **ผสมได้:** จัดกลุ่มตามผู้ซื้อที่จ่ายแพงก่อน แล้วสุ่มในกลุ่ม · ลองแล้วเทียบ PR (หน้า 26)

starter สำหรับแข่ง

ติดตั้งและรัน 4 คำสั่ง

```
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt
cp .env.example .env   # ใส่ MARKET_URL, TEAM_KEY, LLM_*
.venv/Scripts/python main.py --forever
```

--forever ตลาดฝึก: จบเกมแล้วเล่นเกมใหม่ต่อเอง

Ctrl+C แล้วรันใหม่ หลังแก้โค้ดทุกครั้ง

starter สำหรับแข่ง

อ่าน log ให้รู้ว่า agent คิดอะไร

```
offers_sent {"round": "R1", "source": "llm", "n": 8, "seconds": 7.2}
offers_sent {"round": "R2", "source": "llm+rules (1+4)", "n": 5, "seconds": 3.8}
offers_sent {"round": "R5", "source": "rules", "n": 3, "seconds": 30.0}
```

llm
LLM วางแผนครบ

llm+rules
กฎเติมส่วนที่ LLM ไม่ได้ขาย

rules
LLM ไม่ตอบ ใช้ทางสำรอง

วันที่ 18 · 24 · รายละเอียดทุกการตัดสินใจอยู่ใน logs/decisions.jsonl

ทำให้เก่งขึ้น · 11:15

ค้นหา TODO ใน starter

starter เล่นจบเกมได้แล้ว แต่ยังขายแบบเรียบ ๆ · ทุกจุดที่ควรปรับมีคำว่า TODO เขียนไว้ในโค้ด ค้นใน VS Code ด้วย Ctrl+Shift+F

เรื่อง	ทำไมต้องแก้	แก้ที่	ลองเริ่มจาก
① ตัวคูณต่อประเภท	ผู้ซื้อแต่ละประเภทจ่ายไม่เท่ากัน แต่ starter ตั้ง 1.00 ทุกประเภท	strategy.py MARKUP, rule_plan()	ตั้งแต่ละประเภทต่างกัน แล้วขยับตามผลรอบก่อน: ACCEPTED ขึ้นนิด · ABOVE_LIMIT, OUTBID ลง
② เผื่อ noise	ตัวอย่างวัดคลาดได้ ล็อตที่ชิดขอบสเปกอาจโดน SPEC_MISMATCH	strategy.py spec_ok()	เว้นระยะจากขอบสเปกเล็กน้อย ก่อนนับว่าผ่าน
③ ใช้ปฏิทินตลาด	ความต้องการแต่ละรอบไม่เท่ากัน แต่ starter ขายเท่า ๆ กันทุกรอบ	strategy.py sell_budget()	อ่าน calendar จาก /v1/calendar ขายมากรอบเทศกาล · ระวังของค้าง เหลือแค่ 0.80 × TV
④ ให้ LLM รู้มากขึ้น	LLM รู้แค่สิ่งที่อยู่ใน prompt · ตอนนี้เห็นผลแค่ 3 รอบล่าสุด	llm_plan(), memory.py summary()	สรุปผลทั้งเกมแยกตามผู้ซื้อแต่ละราย · บอกปฏิทินรอบถัดไป
⑤ ประเมินแม่นขึ้น	คะแนนประเมินมี 20 คะแนน · ตัวอย่างใกล้ขั้นเกรดทำให้ TV คลาดมาก	appraisal.py estimate_lots()	ดูจาก reveal (notebook ข้อ 4.4) ว่าล็อตแบบไหนคลาดบ่อย

> เลือก 1–2 ข้อพอ · แก้ทีละข้อ แล้วเทียบผลตามหน้าถัดไป · ข้อ ① มักได้ผลเร็วที่สุด

วันที่ 18 · 25 · notebook ข้อ 4.2 มีตาราง "อยากเปลี่ยนอะไร แก้ตรงไหน" · memory.py จำผลข้ามรอบให้แล้ว

ทำให้เก่งขึ้น · 11:15

ทดลองแบบนักวิทยาศาสตร์

แก้แล้วต้อง**พิสูจน์ด้วยตัวเลข**ว่าดีขึ้นจริง · แก้หลายอย่างพร้อมกัน จะไม่รู้ว่าอะไรช่วย อะไรทำให้แย่ลง

① แก้ทีละอย่าง

จดว่าแก้อะไร เช่น ตัวคูณ SPECIALTY จาก 1.00 เป็น 1.10

② เล่นหลายเกม

Ctrl+C แล้วรัน starter ใหม่ · ล็อตและผู้ซื้อสุ่มใหม่ทุกเกม เกมเดียวอาจฟลุก เล่น 2–3 เกม

③ ดูผลจริง

เห็น market_closed แล้ว รันช่อง reveal ใน 5 นาที ก่อนตลาดฝึกเริ่มเกมใหม่

④ เทียบ PR

PR เฉลี่ยหลายเกมสูงขึ้น = เก็บไว้ · ไม่ขึ้น = ย้อนกลับ แล้วลองอย่างอื่น

เกม	แก้อะไร	PR	ประเมินคลาด	สังเกตจาก log
1	starter เดิม	…	…	ประเภทไหนโดน ABOVE_LIMIT
2	starter เดิม	…	…	…
3	SPECIALTY 1.10	…	…	ACCEPTED เท่าเดิมไหม

PR = (รายได้ + 0.80 × TV ของที่ขายไม่หมด) ÷ TV ทั้งหมด

1.00 = ขายได้เท่ามูลค่าจริง · ยิ่งสูงยิ่งดี · เทียบเกมด้วย PR ไม่ใช่คะแนนขาย เพราะคะแนนขายขึ้นกับคู่แข่งในเกมนั้น

วันที่ 18 · 26 · notebook ข้อ 4.4 มีช่องเทียบผลให้แล้ว ตั้งชื่อสิ่งที่แก้ใน CHANGE แล้วรันหลังจบทุกเกม

บ่ายนี้ · 13:00–17:00

ด่านที่ 2: agent แข่งสด · 30% ของคะแนน

13:00–14:30
ปรับ agent ต่อในตลาดฝึก

ราว 14:45
ทีมงานกด Start ตลาดแข่ง

ราว 15:45
จบด่าน 2 · ใช้เวลาราว 55 นาที

ซ้อม 1 + แข่ง 8 รอบ
เปิดรับข้อเสนอรอบละ 4 นาที

ประเมินมูลค่า 10 นาที
ส่งได้ครั้งเดียว ก่อนรอบแรก

ตารางคะแนนซ่อน
หลังรอบที่ 6 ไม่อัปเดต

ห้ามส่งข้อเสนอหรือผลประเมินด้วยมือทุกช่องทาง · LLM ใช้ได้เฉพาะ gateway ของงาน · ผิดกติกา = ตัดสิทธิ์ทั้งการแข่ง (หน้าถัดไป)

วันที่ 18 · 27 · รายละเอียดใน training/กติกาวันแข่ง.md

ผิดกติกาข้อใด ตัดสิทธิ์ทั้งการแข่ง

ผิดกติกา	โทษ
ใช้ LLM อื่นนอก gateway ของงาน ทั้ง cloud และโมเดลบนเครื่องตัวเอง (เช่น Ollama)	ตัดสิทธิ์ทั้งการแข่ง
ส่ง แก้ หรือยกเลิกข้อเสนอและผลประเมินด้วยมือ ในด่าน 2–3	ตัดสิทธิ์ทั้งการแข่ง
ใช้ key ของทีมอื่น หรือให้ key ของทีมกับคนอื่น	ตัดสิทธิ์ทั้งการแข่ง
แลกข้อมูลหรือนัดราคากับทีมอื่นระหว่างแข่ง	ตัดสิทธิ์ทั้งการแข่ง
ตั้งใจยิงคำขอถี่ๆ หรือรบกวนเซิร์ฟเวอร์	ตัดสิทธิ์ทั้งการแข่ง

> gateway ล่ม: agent เล่นต่อด้วยกฎของ starter · ห้ามสลับไปใช้ LLM อื่น · Ollama บนเครื่องตัวเองใช้ฝึกก่อนวันแข่งได้

ก่อนเที่ยง เช็ก 4 ข้อ

- starter รันกับตลาดฝึกครบหนึ่งเกมโดยไม่ล้ม
- .env มี key ของทีมและที่อยู่ LLM gateway
- USE_LLM=0 ก็ยังขายได้ เผื่อ LLM ล่ม
- logs/decisions.jsonl มีบันทึกทุกรอบ